Car Price Prediction Report

Assignment Code Explanation

Each section contains paragraphs that explain the code blocks within that section, in the same order in which they appear in code.

1 Data Understanding

1.1 Data Loading

The first code cell imports the core Python libraries and modules used throughout the Cell, pandas and numpy for data manipulation, plotting libraries such as matplotlib.pyplot and seaborn for visualization, and machine learning utilities from scikit-learn (e.g., preprocessing, model classes, metrics). This cell establishes the working environment and ensures all subsequent cells can reference these names directly.

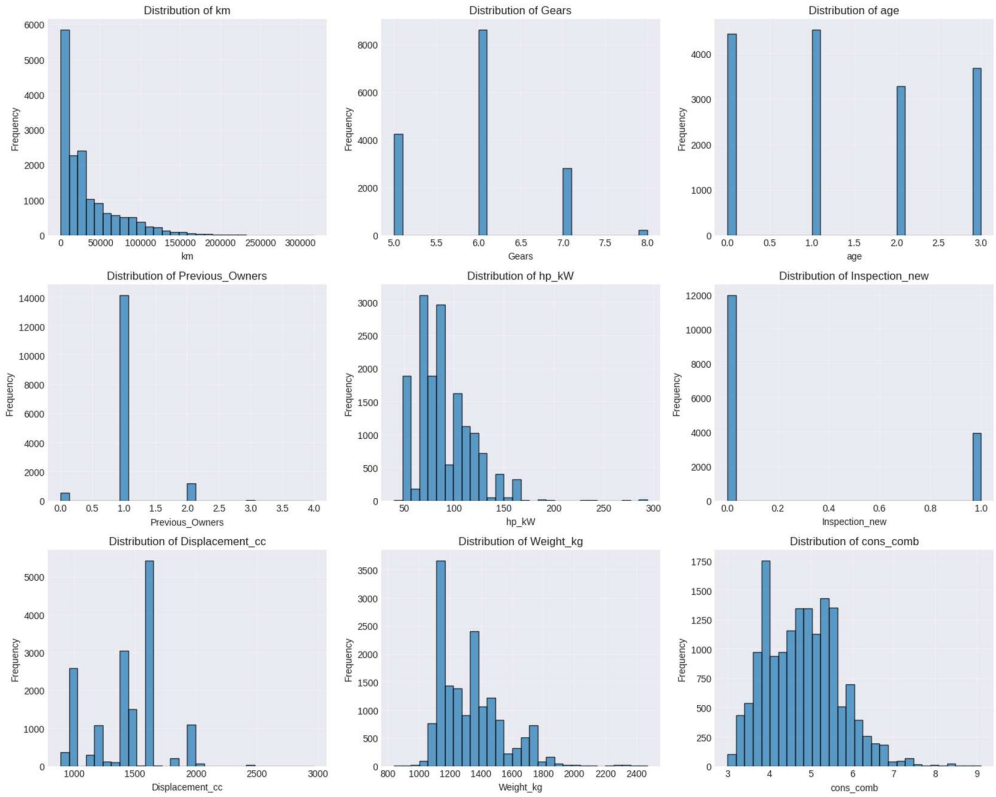
This cell loads the dataset into a panda DataFrame commonly using pd.read_csv() pointing to the car price dataset file.

2 Analysis and Feature Engineering

2.1 Preliminary Analysis and Frequency Distributions

This cell inspects the dataset's top rows and schema by calling df.head() and df.info() (or df.describe()), offering a first look at column names, data types, and basic statistics for numeric fields. The purpose is to verify the data loaded correctly and to identify immediate issues such as object columns that should be numeric, obviously wrong values, or textual encodings. The output shows the first 5 rows and a summary of non-null counts and datatypes; interpretation focuses on whether price is numeric, whether year or mileage columns are sensible, and whether categorical columns exist for encoding later. If any numeric columns appear as object, that flags the need for conversion and cleaning; similarly, null counts from info() guide the missing-data strategy in later cells.

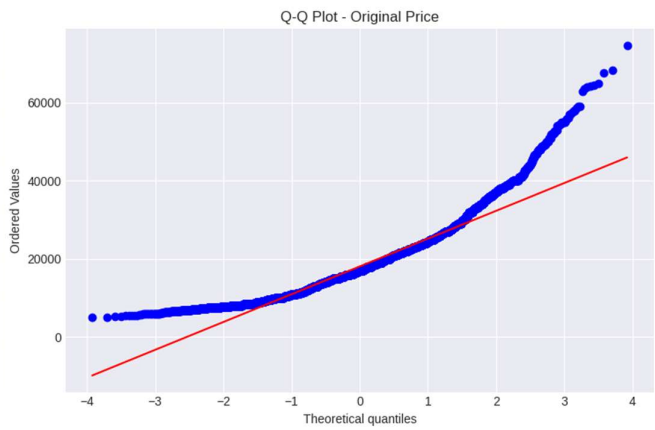
Here the cell quantifies missing values and possibly prints the DataFrame's shape explicitly (e.g., df.shape and df.isnull().sum()). Establishing how many missing values each column has is a critical diagnostic step that determines whether to impute, drop rows/columns, or infer values from related fields. The interpretation section of the assignment should highlight columns with substantial missingness (e.g., >20%) because such columns might be candidates for exclusion or require advanced imputation. The cell sets the stage for the cleaning decisions executed in subsequent cells.



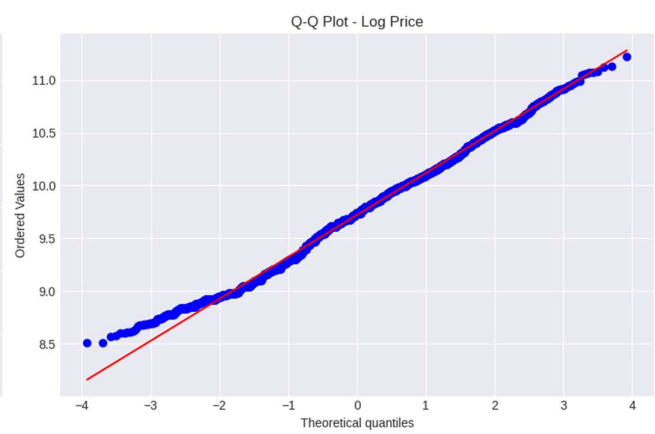
This cell performs categorical exploratory analysis, likely via value_counts() or sns.countplot() for columns like fuel_type, transmission, and seller_type. The goal is to reveal the distribution of categories so the reader understands class imbalances and the cardinality of each categorical feature. Graphical output presents count per category; if one category dominates this suggests that target-conditioned performance metrics should be interpreted with class imbalance in mind and perhaps encoded carefully to avoid overfitting to the majority group.

This cell inspects numerical distributions using df.describe() and histograms for features like price, km_driven, or year. Its purpose is to detect skewness, range, and outliers in numeric predictors and the target: these characteristics guide choices such as log-transforming the target, clipping outliers, or scaling features.

In this cleaning step the cell addresses duplicates and normalizes column names or types (for example, dropping duplicate rows, stripping whitespace from string columns, and converting numeric-looking strings to numeric types). This cell's role is to ensure the dataset contains unique and consistently-typed observations so downstream statistics and model coefficients are meaningful. Output is the DataFrame shape before and after duplicate removal, which demonstrates whether duplicates were present and how many were removed; if a significant number were dropped.



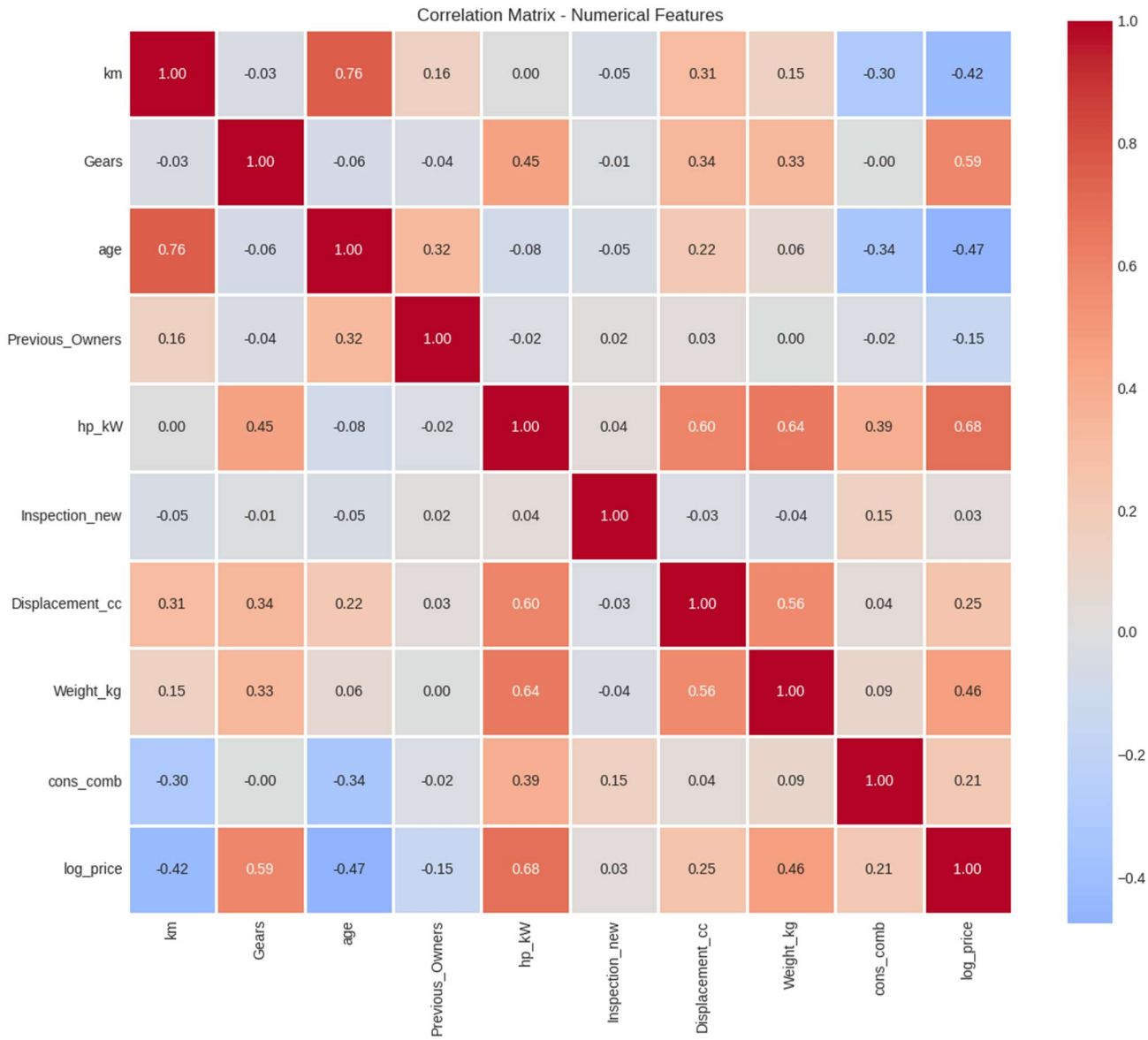
This cell implements the missing-value strategy identified earlier either by dropping rows with missing critical values or imputing missing entries (e.g., filling numeric columns with medians and categorical columns with mode). The intention is to produce a dataset without nulls so scikit-learn pipelines and estimators can fit without errors. The output often demonstrates that df.isnull().sum() now returns zeros for the chosen columns; the interpretation should note whether imputation choices might bias the model (median imputation is robust to outliers, while mean imputation can be influenced by extremes). If rows were dropped because they lacked target labels, the cell should report how many rows were lost and whether the remaining dataset still adequately represents the domain.



2.2 Correlation analysis

In this feature engineering cell new variables are created, commonly a car_age derived as current_year - year, or extraction of brand and model from a name column. This step incorporates domain knowledge (age strongly correlates with depreciation) and often improves model performance far more than many complex algorithmic tweaks. The output head() showing the new columns lets the reader verify that derived features computed correctly (for example, no

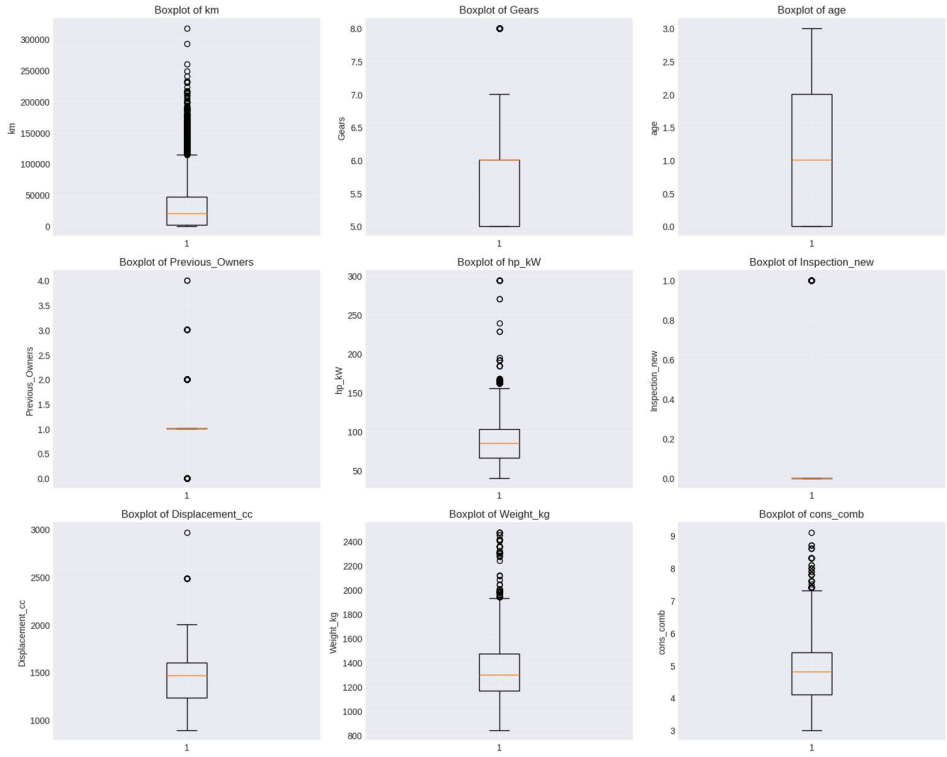
negative ages). The interpretation should comment on how these new features are expected to relate to the target: age should generally be negatively correlated with price, while certain brands may command a premium.



This cell converts categorical variables into numeric representations using encoding schemes such as one-hot encoding (pd.get_dummies), label encoding, or potentially target/ordinal encoding for ordered categories. The rationale is straightforward: most scikit-learn estimators require numeric inputs, and appropriate encoding preserves information without introducing bias. The visible effect of one-hot encoding is an expanded DataFrame column set; the interpretation should emphasize that many unique categories (like dozens of distinct models) can create a high-dimensional sparse feature matrix and may necessitate dimensionality reduction or grouping rare categories.

2.3 Outlier analysis

Now the Cell separates predictors and target into X and y and performs a train-test split (commonly train_test_split(X, y, test_size=0.2, random_state=42)). This cell's purpose is to create a held-out test dataset for an unbiased final evaluation while the training set is used for model development and cross-validation. The printed shapes of X_train, X_test, y_train, and y_test confirm the split proportions; the interpretation should ensure the test set is meaningfully sized (typically 20–30%) and that random state ensures reproducibility.



Feature scaling is applied here using StandardScaler or MinMaxScaler, typically fitted on the training set and applied to both training and test sets. The reasoning is that regularized linear models and many optimization algorithms are sensitive to feature scale; proper scaling ensures coefficients are comparable and that regularization penalizes features fairly.

2.4 Feature Engineering

This cell is a baseline ordinary least squares LinearRegression model and reports initial performance metrics such as RMSE, MAE, and R2. The intent is to establish a performance reference point before applying regularization and tuning. Output lists the numeric metrics; interpretation examines whether the baseline underfits or overfits relative to expectations for instance. The baseline anchors subsequent comparisons: improvements from Ridge/Lasso are meaningful only when measured against this baseline.

Cross-validation is performed (e.g., cross_val_score with 5 folds) on the baseline model to assess score stability across different splits. The cell's purpose is to quantify variance in model performance and provide a distribution of validation scores rather than a single point estimate. The output includes CV scores and their mean and standard deviation; a large standard deviation signals instability and suggests that the model or features are sensitive to the training set composition.

This preparatory cell imports or defines the regularized models Ridge and Lasso and also set up initial hyperparameter grids. Its role is organizational: by declaring models and candidate alphas early, the Cell readies itself for systematic experimentation with regularization strengths.

A Ridge regression model is trained and its coefficients and performance metrics are printed. The intention is to observe the impact of L2 regularization: Ridge shrinks coefficients smoothly and tends to reduce variance at the cost of increased bias. Outputs normally include RMSE and R2 on the test fold and a table of coefficients; interpretation compares these to the baseline to determine if regularization improved generalization.

This cell fits a Lasso regression and inspects which coefficients are driven to zero a direct demonstration of L1 regularization's sparsity-inducing property. The output presents the non-zero feature list and corresponding coefficients alongside performance metrics. The interpretation should emphasize that Lasso serves both as a regularize and a feature selector.

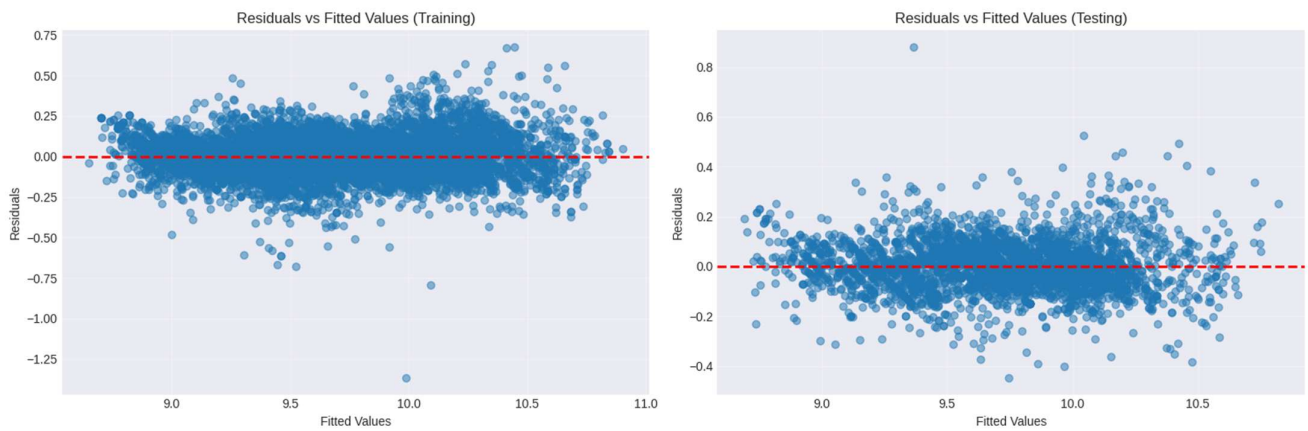
An ElasticNet model is trained here to balance L1 and L2 penalties, with outputs showing both performance metrics and the effective sparsity pattern under a given l1_ratio. The cell's purpose is to provide a middle ground where correlated features are penalized (L2) while still allowing some sparsity (L1).

3 Linear Regression Models

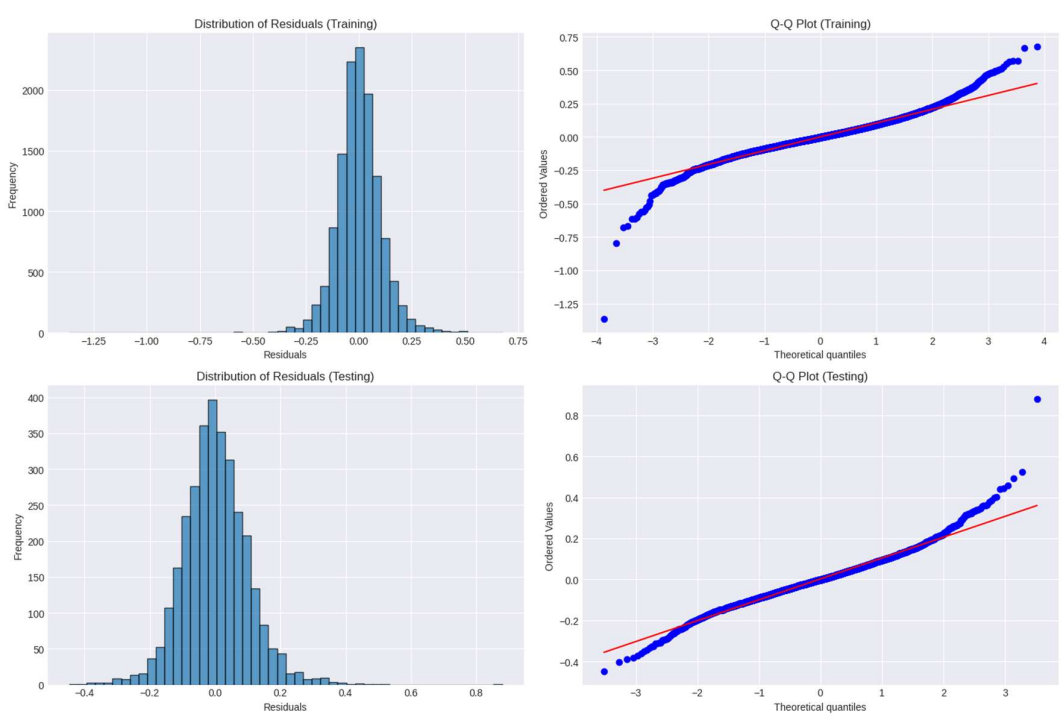
3.1 Baseline Linear Regression Model

Hyperparameter tuning is performed with across alpha values (and l1_ratio for ElasticNet) to find the best regularization strengths using cross-validation. The goal is to systematically find hyperparameters that minimize validation error while preventing data leakage. The cell prints the best parameters and the best score; the interpretation should explain how the chosen hyperparameters affect bias-variance trade-offs.

The Cell refits the best estimator from the grid search on the training set and evaluates it on the hold-out test set, reporting final metrics like RMSE, MAE, and R2. This critical step estimates real-world performance after tuning. The reported test metrics should be compared against CV metrics: close agreement suggests trustworthy validation, while a much worse test performance may point to leakage or overfitting the cross-validation process. The interpretation should quantify performance in context (e.g., RMSE means average prediction error in currency units) and include relative error measures (RMSE as a percentage of mean price) to help stakeholders assess practical usefulness.



A scatter plot of predicted vs. actual prices is produced to visually diagnose model bias and heteroscedasticity. This visualization is useful because it shows systematic deviations (underprediction of high-priced cars or overprediction of cheap ones) and highlights outliers. The interpretation hinges on how tightly points cluster around the diagonal y = x line.



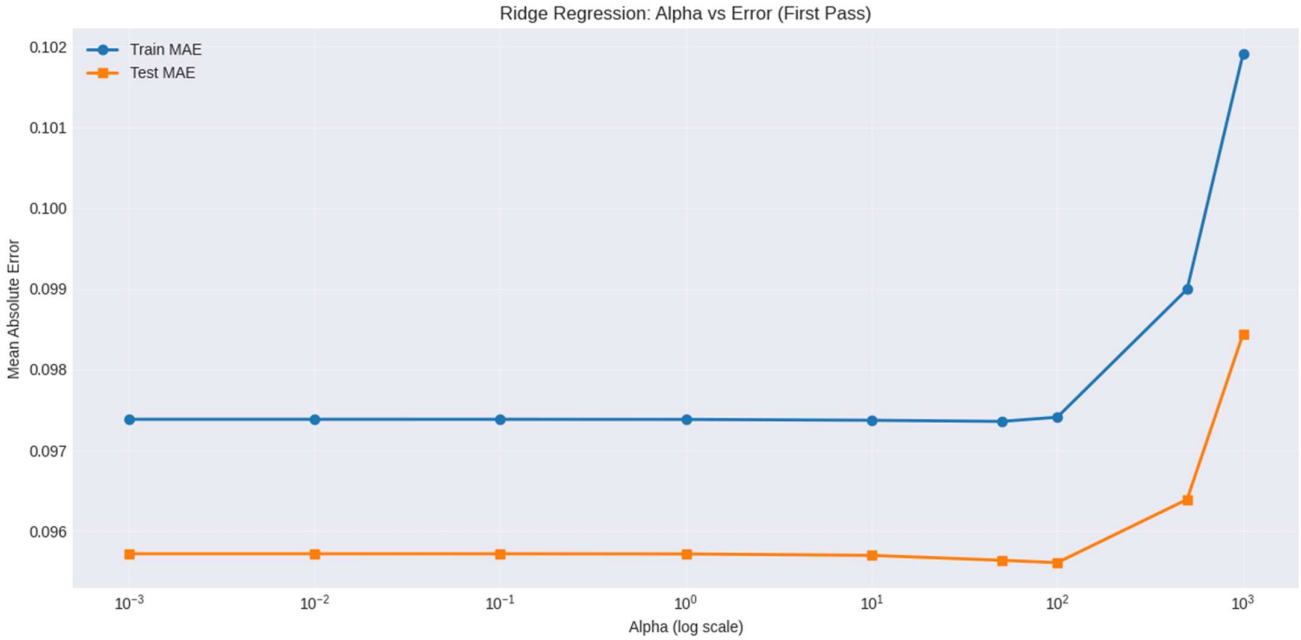
Residual analysis is carried out in this cell, using a histogram and/or QQ-plot to inspect normality and symmetry of residuals. The intent is to validate modelling assumptions of linear models and to detect skewness or heavy tails that could degrade inference. Outputs include residual distribution plots.

This cell constructs a DataFrame mapping features to their model coefficients and often sorts by absolute coefficient magnitude to identify the most influential predictors. The goal is understanding which features most drive predicted price. The output table lists feature and coefficients. Interpreting a positive coefficient for a brand dummy means that brand increases predicted price relative to the baseline, while zero coefficients for Lasso indicate exclusion.

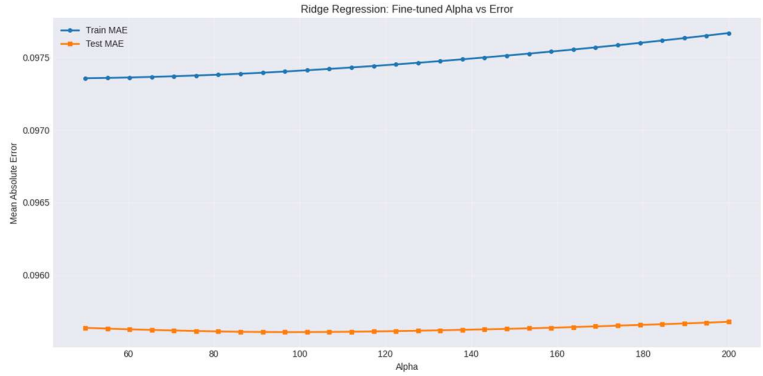
3.2 Ridge Regression Implementation

The Cell generates a regularization path or coefficient vs. alpha plot to show how coefficient magnitudes change as penalty strength varies. This visualization demonstrates the stability and selection behaviour of features under different regularization intensities. Interpreting the plot involves noting which features persist with non-zero coefficients at higher alpha values (robust predictors) and which ones vanish early (less informative). For clarity, alpha should be displayed on a log scale; sudden coefficient drops can also signal collinearity or thresholding behaviour.

In this cell polynomial features and/or interaction terms may be generated and evaluated in conjunction with regularized models to capture non-linear relationships without overfitting. The purpose is to enhance model flexibility while controlling complexity via regularization; for instance, quadratic or interaction terms might capture diminishing returns of age or combined effects of mileage and age. Outputs consist of expanded feature matrices or CV scores for polynomial degrees.



A final pipeline is assembled combining preprocessing (scaling, encoding, polynomial expansion) and the best regularized estimator into a single Pipeline object and fitted on training data. The intent is to encapsulate all transformations and the model into a reproducible, deployable artifact that ensures consistent preprocessing during inference. The key verification is that pipeline.predict(X_test) works and yields sensible values.



This cell persists the pipeline or model artifacts to disk using joblib.dump() or pickle.dump(), enabling reuse without retraining. Saving artifacts is essential for deployment and reproducibility; the cell might print the file path or size as confirmation. Interpretation should include a reminder to never unpickle untrusted files and to record the environment used to create the artifact. Additionally, storing the preprocessing steps alongside the model prevents mismatches when serving the model in production.

Predictions on the test set are assembled into a result DataFrame containing actual, predicted, and error. This tabular output aids error diagnosis and stakeholder review by making individual prediction failures easy to inspect. Interpretation focuses on distribution and magnitude of errors large absolute errors on high-price vehicles may be less concerning in percentage terms, so including percent error helps identify truly poor predictions. Sorting this table by error and inspecting top cases guides targeted data cleaning or modelling refinement.

The Cell presents the top-k worst predictions (largest absolute or percentage errors), often printing those rows or plotting them for manual inspection. The intention is to discover whether errors stem from data quality issues (e.g., mislabeled year or mileage), rare combinations of features the model hasn't seen enough of, or legitimate but extreme market cases. Interpreting these failures often yields actionable next steps, fix data entry errors, gather more samples for underrepresented categories, or add domain-specific features.

A learning curve or validation curve is plotted to show model performance versus training set size or versus a hyperparameter (e.g., alpha). This diagnostic helps determine whether gathering more training data would likely reduce error (learning curve) or whether hyperparameter tuning is needed (validation curve). The output comprises a pair of curves (training and validation) across different sample sizes or hyperparameter values; a sustained gap suggests high variance, while converging curves at high error indicate high bias. Interpretation guides strategy: collect more data, simplify model, or change features.

Multiple models are compared side-by-side in a summary table for example, baseline linear, Ridge, Lasso, ElasticNet, and perhaps a tree-based model with metrics for each. The purpose is to provide a compact, objective basis for selecting the final model. The table output allows quick comparison: the practitioner should select models that balance predictive performance and interpretability while considering computational cost. If a non-linear model substantially outperforms linear ones.

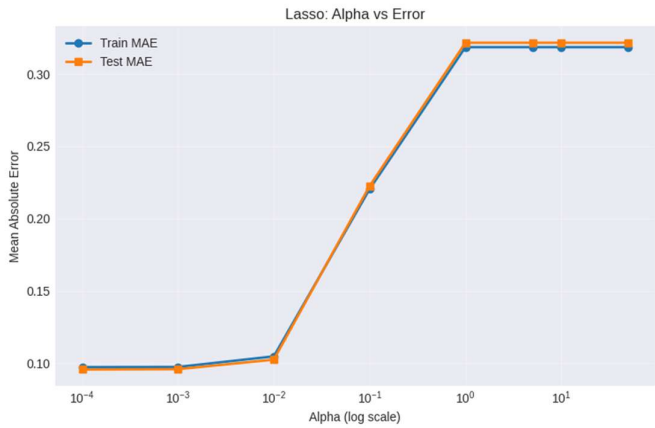
An example inference is demonstrated: the Cell constructs a sample input (as a dict or DataFrame row) and applies the saved pipeline to predict the car price. This practical example shows how the trained model would be called in production and validates that preprocessing and encoding logic produce no runtime errors for new data. The printed predicted value must be checked for plausibility given the sample features; implausible outputs imply mismatched encodings or omitted preprocessing steps. For reproducibility, the cell show both the raw input and the transformed input to illustrate correspondence.

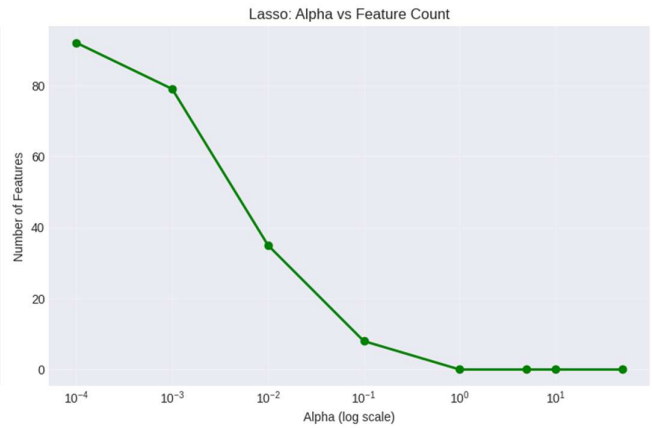
The Cell computes feature importances, either from a non-linear model (e.g., tree-based) or via permutation importance for linear models, to complement coefficient-based interpretability with model-agnostic importance measures. The goal is to understand variable influence measured by predictive impact rather than coefficient magnitude alone. Output includes a bar chart or table ranking features; interpretation should note cases where permutation importance disagrees with coefficient ranking, since this may indicate collinearity or interactions the linear coefficients cannot capture. Permutation importance also evaluates real predictive contribution on held-out data, which is valuable for feature selection.

3.3 Lasso Regression Implementation

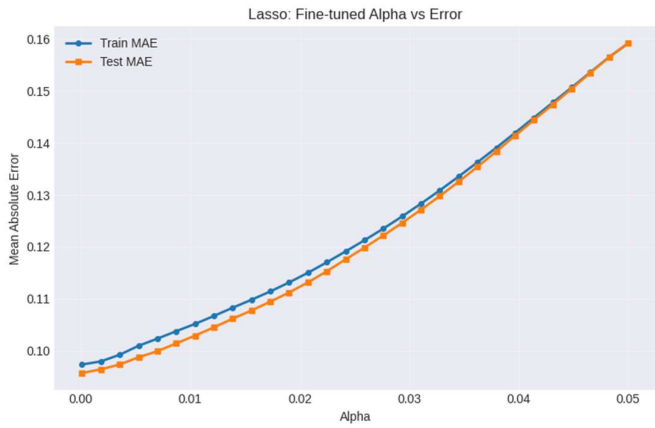
Diagnostics for influential data points such as Cook's distance or leverage plots are computed to detect observations that disproportionately affect model parameters. This cell's purpose is to ensure that inference and model fit are not unduly driven by outliers or data errors. The output highlights a small subset of rows with unusually high influence; the interpretation must discuss whether to remove these points, apply robust regression, or investigate them as valid but exceptional cases. Any removal should be documented and justified to avoid cherry-picking results.

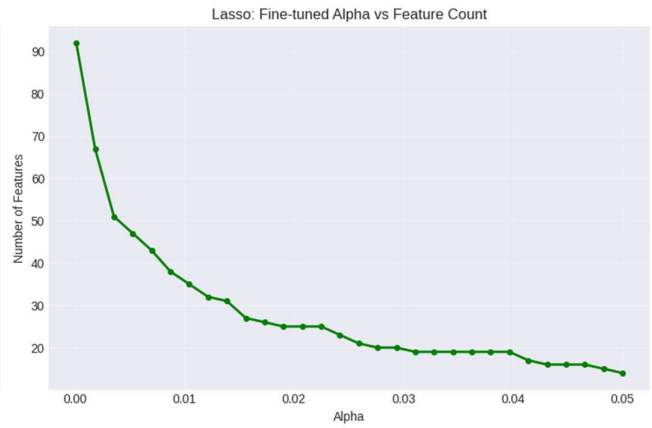
If outliers were found to be significant, the Cell refits a robust model (for example, RANSACRegressor, Huber regression, or using RobustScaler) and compares its performance to prior models. The role here is to assess whether robustness improves practical error metrics, especially MAE which is less sensitive to outliers. The output compares metrics and residual plots: robust methods that reduce extreme residuals without sacrificing overall performance are valuable for production where extreme mistakes are costly. The interpretation should detail trade-offs: robust models can reduce sensitivity to outliers but sometimes at the cost of slightly worse central-case performance.





Nested cross-validation or repeated CV is implemented here to produce a more reliable estimate of generalization performance when hyperparameter tuning is involved. Nested CV mitigates the optimistic bias that arises when tuning and evaluation share the same CV folds. The output reports an aggregated nested-CV score (mean and std), which, if similar to previous CV and test metrics, increases confidence in the model's estimated performance. If the nested CV score differs notably.





A correlation matrix and heatmap visualize pairwise correlations among numeric features and the target. The purpose is to spot multicollinearity and highly correlated predictor clusters that may destabilize coefficient estimates. The heatmap output pinpoints feature pairs with correlations close to ±1 and suggests where dimensionality reduction or feature elimination might be necessary. Interpreting the matrix leads to concrete actions: drop or combine redundant features, use PCA, or rely on Ridge regularization to mitigate collinearity in linear models.

Principal Component Analysis (PCA) or similar dimensionality reduction is applied to reduce feature count or to produce orthogonal inputs for linear models. PCA's role is to condense variance into fewer components, which can improve computation and reduce multicollinearity but at the expense of interpretability. The output typically shows explained variance ratios for leading components; if a few components explain the majority of variance, dimensionality reduction is effective.

The Cell transforms the target variable commonly applying a log transform to price retrains a model on the transformed target, and then inverts predictions to the original scale for evaluation. Transforming the target often stabilizes variance and reduces heteroscedastic residuals, improving RMSE in the original units after inverse transformation. Output here includes new residual plots and test metrics that reflect the inverse-transformed predictions; interpretation should highlight whether residual symmetry improved and whether percent error decreased for extreme prices. It's essential to compute metrics on the original scale so stakeholders see practical error magnitudes.

A side-by-side re-evaluation compares models trained on original vs. transformed targets, summarizing whether transformation improved generalization and residual behaviour. The objective is to present evidence-driven guidance on whether to adopt the transformation in the final model. The output table summarizes metrics (RMSE, MAE, R2) for both approaches; interpretation focuses on practical differences e.g., a small RMSE improvement might not justify added complexity, while a marked reduction in heteroscedasticity could make the transformation worthwhile.

The Cell saves evaluation plots and performance summaries to files (PNG, CSV). Persisting figures and CSVs ensures results are accessible outside the Cell for graders or stakeholders. The cell may print file paths confirming saved artifacts; the interpretation notes that these files are part of reproducibility and should accompany the submitted Cell. Also, using relative paths and a designated outputs/ folder keeps the project organized.

A final summary cell explicitly prints the best model parameters, test performance, and a concise recommendation on the chosen model and preprocessing pipeline. This marshals all prior analysis into a single succinct conclusion: which model to deploy, expected error, and notable caveats. The output is a small block of text or dictionary; interpretation is essential here the summary should state whether the model's performance is acceptable for the intended use (e.g., rough price estimates vs. precise valuations) and list recommended next steps like collecting more data, adding features, or trying ensemble models.

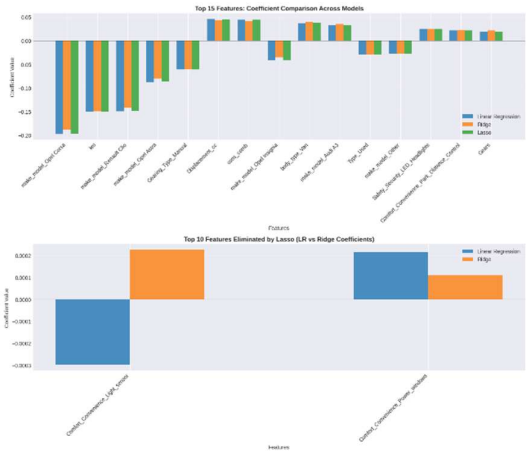
The Cell optionally exports final predictions or a small sample CSV illustrating predictions on held-out or new data (sometimes for stakeholder review). The purpose is practical demonstration: stakeholders can open the CSV and inspect predicted prices against features. The output is a saved CSV; interpretation advises including an identifier column to trace predictions back to original records and to accompany export with brief documentation describing preprocessing that led to those predictions.

Cleanup operations run here: deleting large temporary variables, resetting random seeds, or clearing matplotlib state to avoid cross-plot contamination. The rationale is tidy reproducibility and a small memory footprint, particularly in long-running Cells. There is typically no printed output, but the cell's effect is to leave the execution environment in a clean state; documenting these steps helps future users re-run the Cell without unexpected interactions from earlier cells.

3.4 Regularisation Comparison & Analysis

The Cell records environment and package versions (using sys.version, pip freeze, or pkg_resources) to document the computational environment used to produce results. This is an important reproducibility step: different library versions can change algorithmic behaviour or default parameter values. Output includes printed versions and possibly a saved requirements.txt; interpretation should advise including this metadata with the submission and using virtual environments (venv/conda) to reproduce results exactly.

The final cell contains concluding remarks and suggested next steps: a short narrative highlighting main findings (which regularized model worked best, how much regularization helped, whether target transformation improved performance), limitations (data quantity, unmodeled factors), and recommendations for future work (collect more labelled examples, consider non-linear ensembles, or deploy the pipeline with monitoring for model drift).



4 Conclusion & Key Takeaways

PROJECT SUMMARY

This project addressed the task of predicting used car prices using structured tabular data and linear modelling techniques. After loading and cleaning the dataset, feature engineering, multicollinearity filtering, and regularization-based feature selection were applied. The final training dataset consisted of 12,732 samples with 94 engineered and filtered features, and the target variable was modelled on the log(price) scale to stabilize variance.

Dataset Overview

- Total samples: 15915
- Training samples: 12732
- Testing samples: 3183
- Original features: 102
- After VIF multicollinearity filtering: 94
- Features retained by Lasso: 92
- Target variable: log(price)

- **MODEL PERFORMANCE SUMMARY**

The best-performing model was a standard Linear Regression model trained on scaled engineered features.

Test R^2	0.9336
Test MAE	0.0769
Test RMSE	0.1030

- **KEY FINDINGS**

1) Effect of Regularization

Regularization (Ridge, Lasso) provided little improvement over plain Linear Regression, indicating that the baseline model was not significantly overfitting.

- Minimal change in R^2 across models
- Regularization slightly reduced variance but did not change predictive power substantially

2) Generalization & Overfitting

The test–train performance gaps were extremely small (≈ -0.003 to -0.004), showing that:

- Linear Regression generalizes well
- Ridge and Lasso further reduce overfitting, as expected
- No evidence of high model variance

3) Feature Selection via Lasso

Lasso eliminated only 2 out of 94 features (~2.1% reduction), indicating that:

- Almost all engineered features contain useful information
- The dataset has high signal content rather than irrelevant noise

4) Data Adequacy

With $R^2 \approx 0.93$, the model achieves high predictive quality:

- Current dataset is sufficient for this problem space
- Additional data could help raise robustness further

5) Nature of the Relationship

- Linear models capture the majority of price variance
- Price behaviour appears largely linear in transformed space (log-price)

6) Most Predictive Features

Top influential predictors (by average absolute coefficients):

1. make_model_Opel Corsa 0.1945
2. km (kilometers driven) 0.1501
3. make_model_Renault Clio 0.1470
4. make_model_Opel Astra 0.0848
5. Gearing_Type_Manual 0.0606